

生成式软件开发

生成式软件开发-11

- 从大众开发到专业开发

从大众到专业开发：什么时候你需要考虑更多的事情

- **当你不再满足于一次性使用时**：数据如何存储、如何保证一致性、如何确保不丢失
- **当你想把这个软件分享给同事同学时**：如何在本地或者云服务器上部署你的软件
- **当你想确保系统性能和可靠性时**：如何支持几百到几千用户稳定可靠访问且能快速响应
- **当你想持续扩展和改进你的软件时**：如何根据用户建议持续扩展功能、修复缺陷、改进质量

我们还缺些什么-1

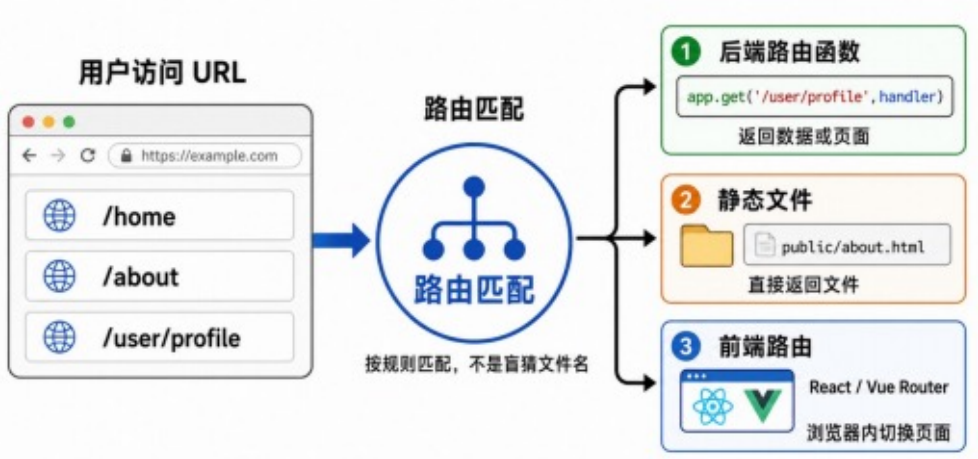


不理解模态窗口、弹窗、抽屉等界面层级：页面其实弹出了登录框、确认框、侧边抽屉或遮罩层，但不知道这些属于独立的界面状态，只会跟AI说“界面没有变化”“页面空白”“按钮没反应”。 AI很难判断是弹窗被遮住、样式层级错误，还是功能根本没触发。



不会描述“复现步骤”：遇到问题只说“它坏了”“点不了”，但不说明自己先点了哪个按钮、输入了什么内容、在哪个页面发生。AI缺少操作路径，只能猜测，容易改错位置。

我们还缺些什么-2

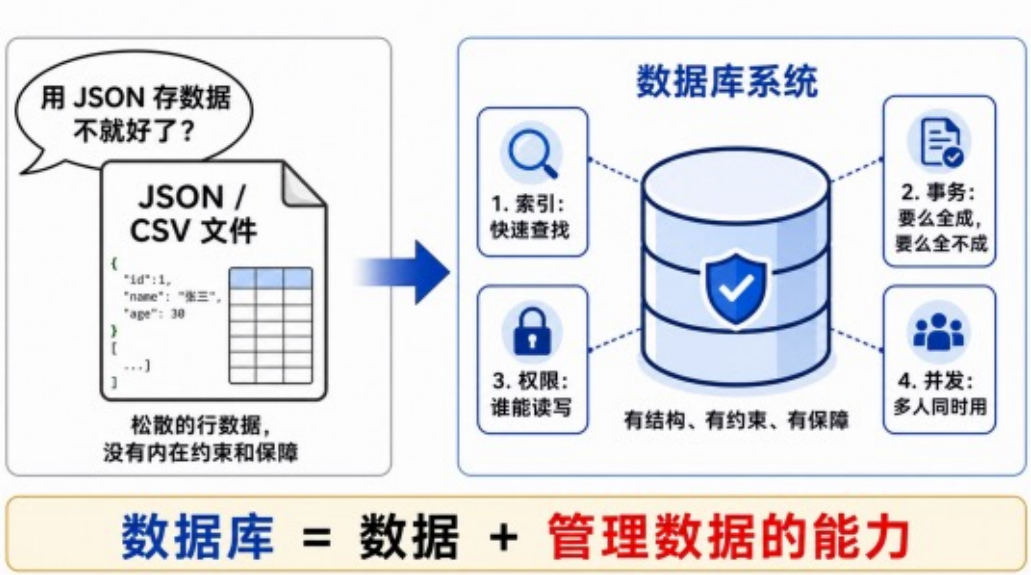


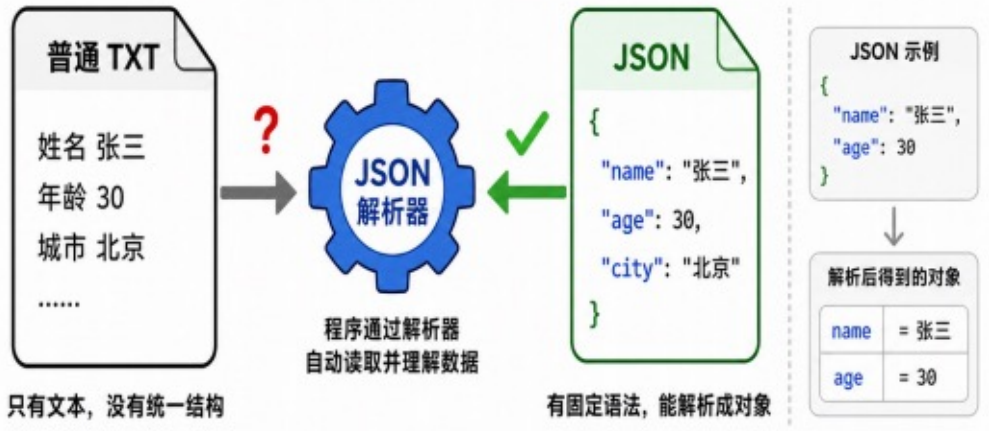


不理解URL路由与代码文件的对应关系：在项目中写了多个页面（/home，/about，/user/profile），但不知道 这些路径对应后端代码的哪一段（是静态文件？还是路由函数？还是前端路由？）。尝试修改某个路径的行为时，在所有文件里盲目搜索。

不会区分空状态、加载状态和错误状态：页面显示“暂无数据”、 一直转圈、白屏、500报错，看来都叫“页面空白”。但对AI来说，这几种情况对应的原因完全不同：可能是数据库没数据、接口卡住、前端渲染报错或服务器异常。

我们还缺些什么-3

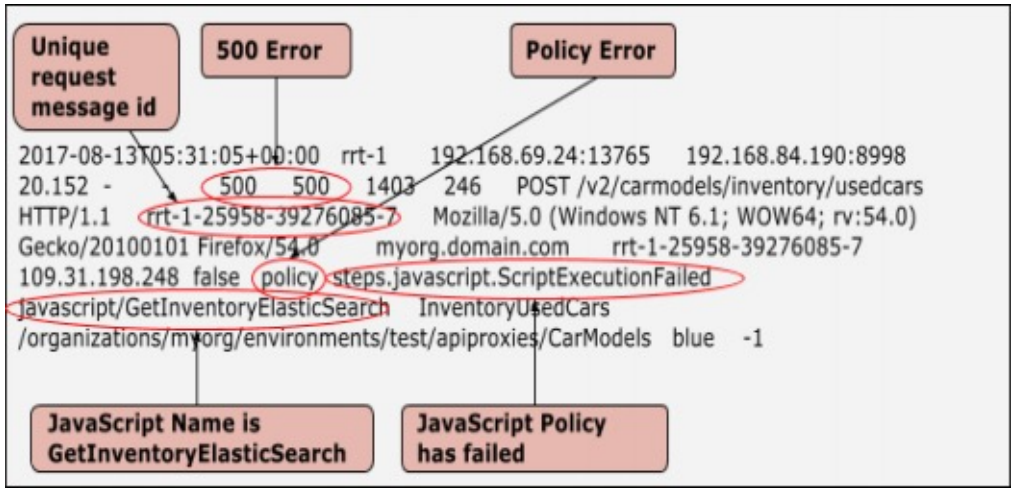


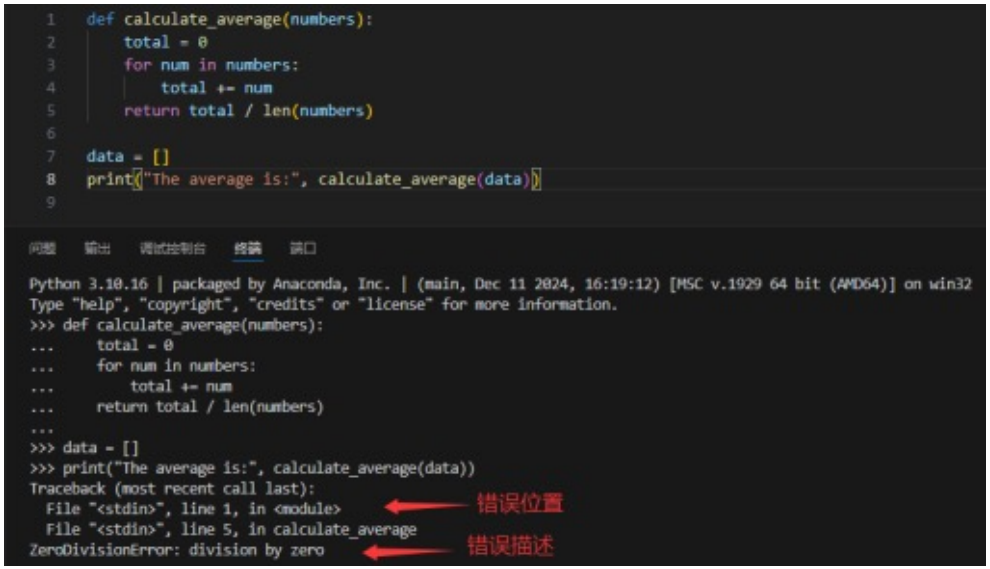


对数据库的理解停留在“明文文件”：认为数据库一定就是可以直接打开看的JSON文件或CSV表格。不理解为什么需要数据库，认为“用JSON存数据不就好了？”不知道索引、事务、权限等概念。

不理解JSON的格式与作用：知道JSON像一种“有括号的文本”，但不知道为什么程序能自动读取它，而普通TXT不行。误以为JSON必须手写得非常漂亮（缩进严格）。

我们还缺些什么-4



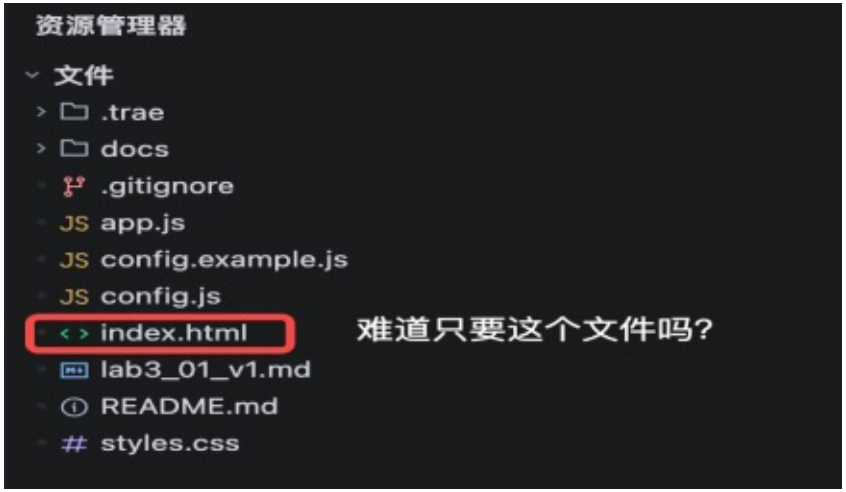


不查看运行日志：后端报500错误，不知道去云服务平台的后台看实时日志；前端报错不知道打开浏览器开发者工具的Console和Network面板。只会把页面显示的错误直接贴给AI。

不看报错信息/不会向AI提供报错信息：程序报错后，不看红字内容，直接复制整个屏幕给AI说“帮修”。 或者只 看最后一行Error，不看Traceback定 位到具体行号。

我们还缺些什么-5





不理解工作区与文件树逻辑：打开Trae等IDE后，不知道侧边栏的文件树就是Workspace的内容，不清楚“打开文件夹”与“打开文件”的区别。有时AI建议修改某个文件，不知道去Workspace里找到它，或误以为关闭IDE标签页就会删除硬盘上的文件。

混淆“编辑器内预览”与“真实部署”：认为只要在IDE里点“运行/预览”，作业就算完成了。提交作业时只发来一个“index.html”文件，理由是“在我自己电脑上双击就能打开，为什么不行？”。

从大众开发到专业开发的能力跨越

- 信息与计算思维
  - ✓ 问题的结构化分析与描述
  - ✓ 软件使用模式的准确表达
- 计算机系统知识
  - ✓ 软硬件技术栈及各层次上的系统原理
  - ✓ 软件的部署和运行机理
- 软件工程能力
  - ✓ 软件设计与编程能力
  - ✓ 软件开发工具理解与使用

问题的结构化分析与描述：场景分析与描述

- 以一种直观和形象化方式描述和谈论待开发的软件系统将如何与各类用户打交道
- 场景（scenario）表示一组参与者（actor）与待开发系统之 间的一系列交互步骤
  - ✓ 参与者是待开发系统边界之外的人（如图书管理员）、设备（如刷卡机）或其他系统（如在线支付系统）
  - ✓ 交互序列有明确的开始和结束，构成一个相对完整的整体，其目的是满足相关涉众的一个或一组目标
  - ✓ 存在一些失败场景，其中所追求的目标最终并未实现
- 具有相同目标且交互过程相似的一组场景可以被组织为一个用例（use case）
  - ✓ 存在一个在大多数情况下会发生且满足目标的主场景
  - ✓ 还包括与主场景有所差异的可能场景以及异常处理场景

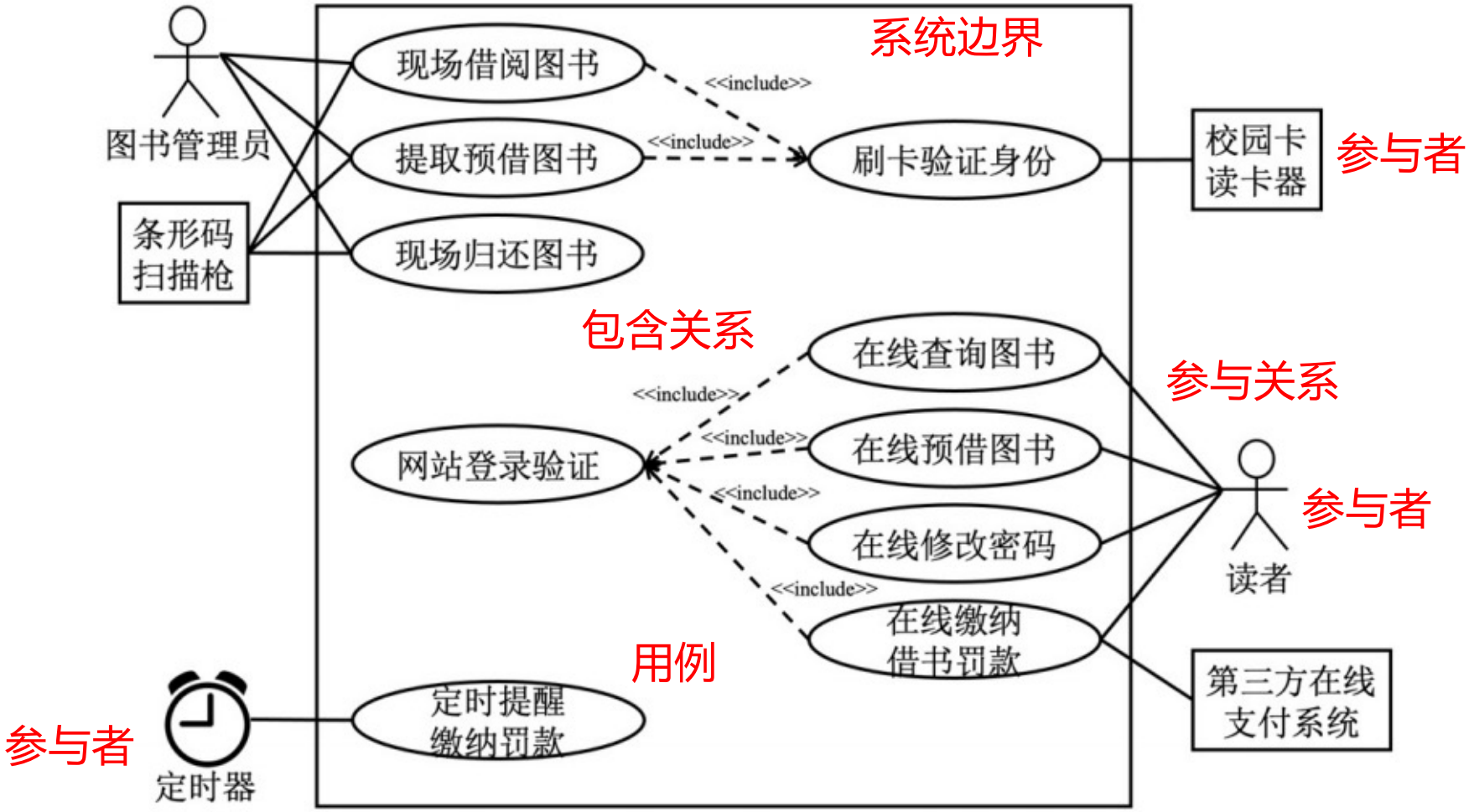
用例参与者的类型

- 由人来扮演的角色
  - ✓ 需要与目标系统交互的各类用户，代表一种抽象的角色
  - ✓ 例如食堂消费者、食堂窗口工作人员、 图书馆读者等
- 外部设备
  - ✓ 与目标系统交互的各种外部设备，包括传感器和执行器
  - ✓ 外部设备拓展了软件系统感知和操控物理世界的能力
  - ✓ 例如条码扫描枪、温度和湿度传感器、机械臂等
- 外部系统
  - ✓ 与目标系统交互的其他外部软件系统，可以提供所需的服务或信息或者请求目标系统提供所需的服务或信息
  - ✓ 例如提供在线支付服务的第三方在线支付系统
- 定时器
  - ✓ 一种特殊的参与者，会定时触发预定义的处理逻辑

与参与者的交互接口

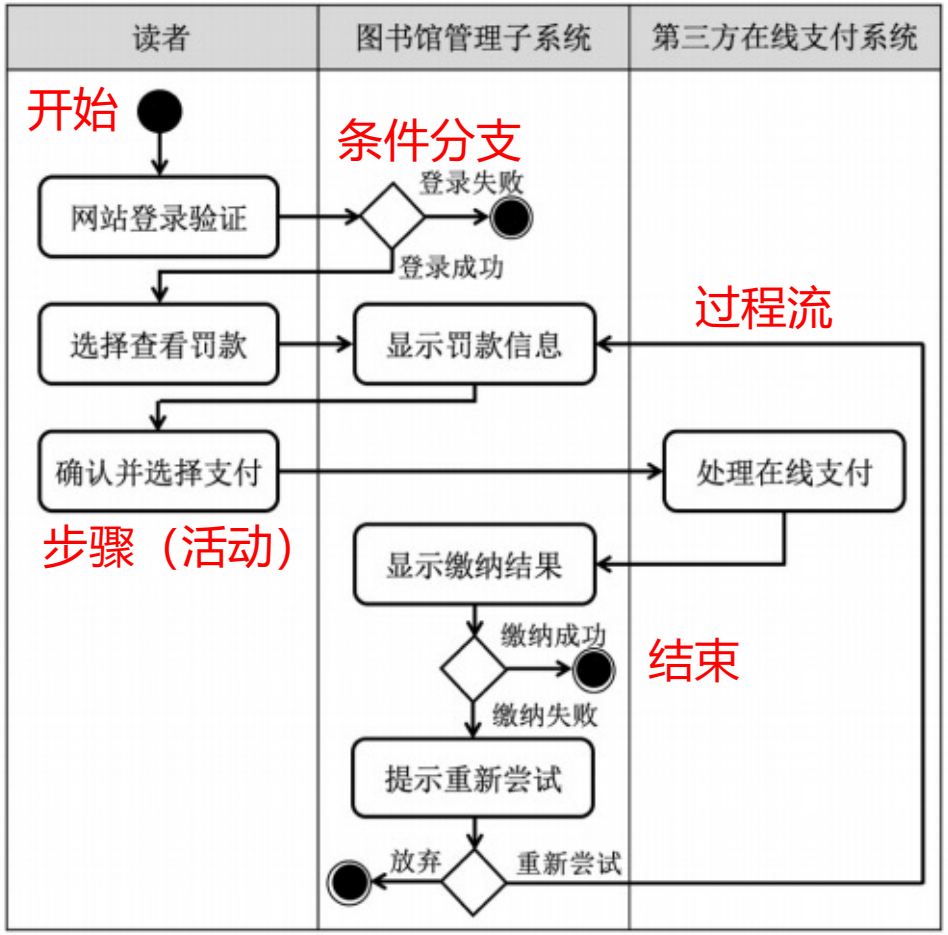
- 除了定时器外其他参与者都位于目标软件系统边界之外，他们（它们）与目标系统进行交互，但并不属于目标系统的一部分
- 目标系统通过开发各种接口（interface）来实现与这些参与者的交互
  - ✓ 与外部用户角色交互的用户接口（又称用户界面，简称UI）
  - ✓ 与外部设备交互的硬件接口（如访问校园卡读卡器的接口）
  - ✓ 与外部系统交互的软件接口（如第三方在线支付服务接口）
- 目标软件系统中所实现的这些接口属于系统的一部分，而这些接口交互的对象（例如用户、硬件、外部系统）则属于系统边界之外

用例概览描述（UML用例图）



用例详细描述（UML泳道图）

泳道（交互方）



针对“在线缴纳借书罚款”用例的详细描述，展开描述用例内部的交互序列

UML泳道图是活动图的一个变体，此外还可以用UML顺序图来描述

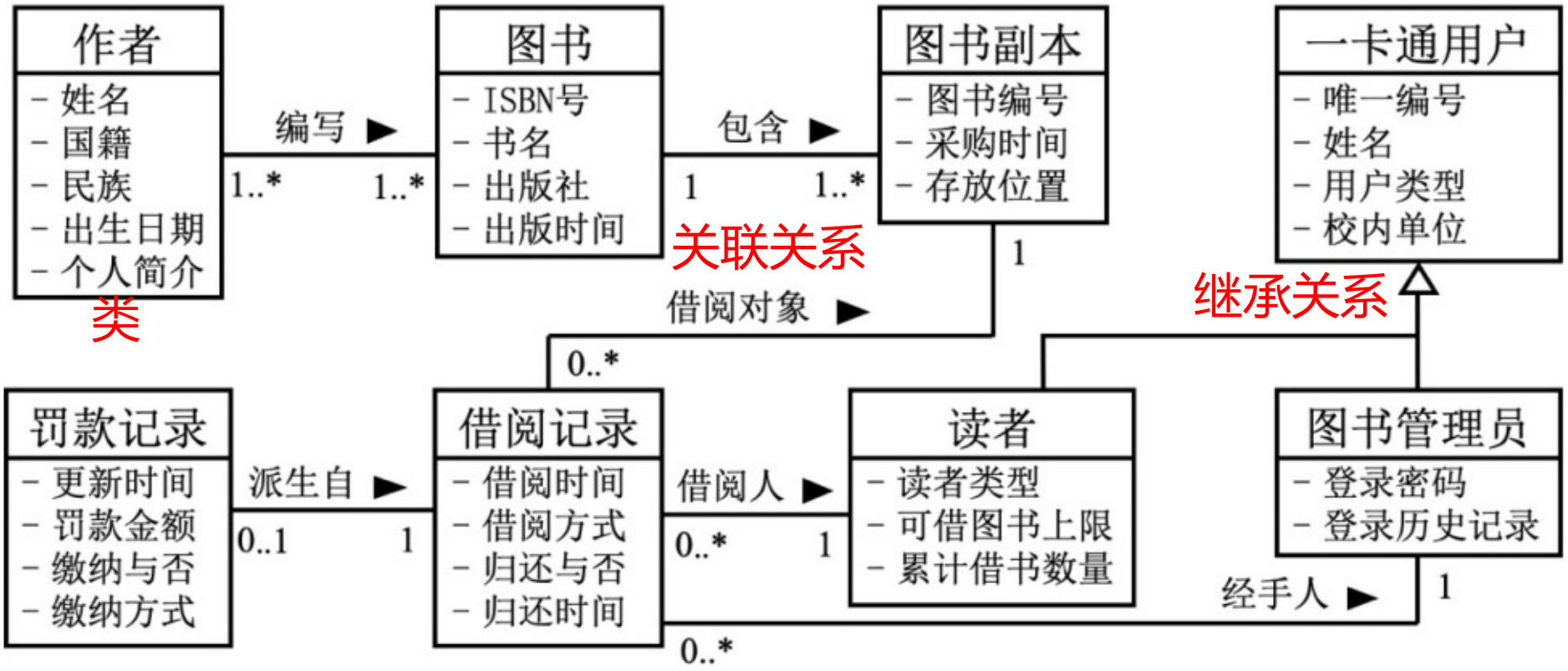
问题的结构化分析与描述：类的分析与描述

- 关注于对象属性以及相互之间的关系
- 理解场景、功能等需求描述的基础，也是设计阶段考虑数据结构（如数据库表）设计的基础
- 基于面向对象的类分析方法
  - ✓ 类是属性和操作的统一封装体，代表着同一类对象或实体的共性抽象
  - ✓ 类包含一组所有对象或实体都具有的属性和操作
  - ✓ 类之间存在多种不同类型的关系：继承（Inheritance）、聚合（Aggregation）、关联（Association）、依赖（Dependency）等
- 关心分析类：不考虑设计和实现方案的情况下分析需求的过程中可以识别出来的类

不同类型的分析类

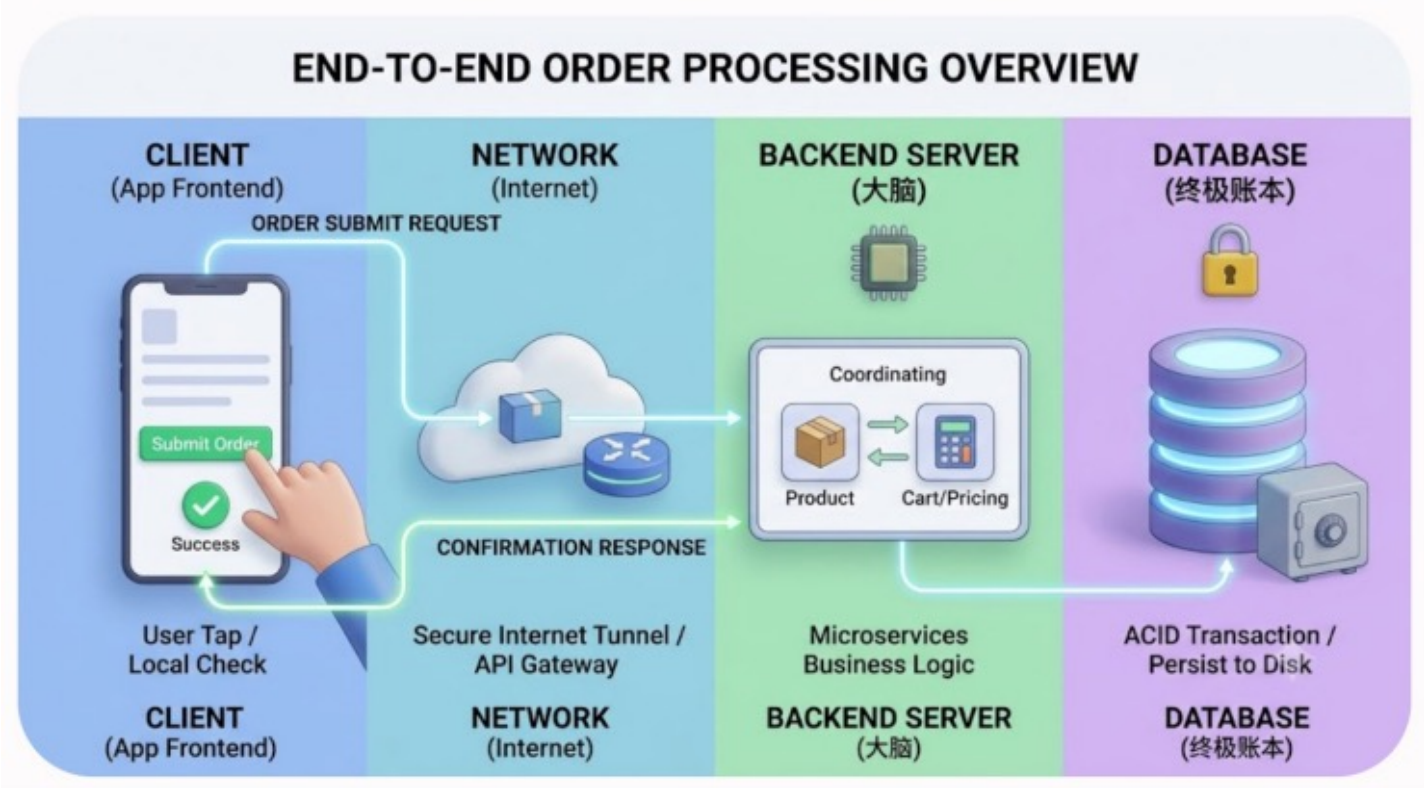
- 外部实体：产生或使用目标软件系统信息的外部实体，例如人员、设备、其他系统
- 事物：目标软件系统所管理的各种事物，例如订单、发票、商品
- 事件：发生的事件，一般具有时间戳，例如报警、来访
- 角色：由人扮演的各种角色，例如读者、管理员
- 组织单元：人类社会的各种组织单位，例如公司、部门、学校、院系
- 场地：现实世界的场地空间，例如房间、楼宇
- 结构：各种结构体，例如车辆、电器

类模型描述（UML类图）



校园一卡通系统图书馆管理子系统部分类模型

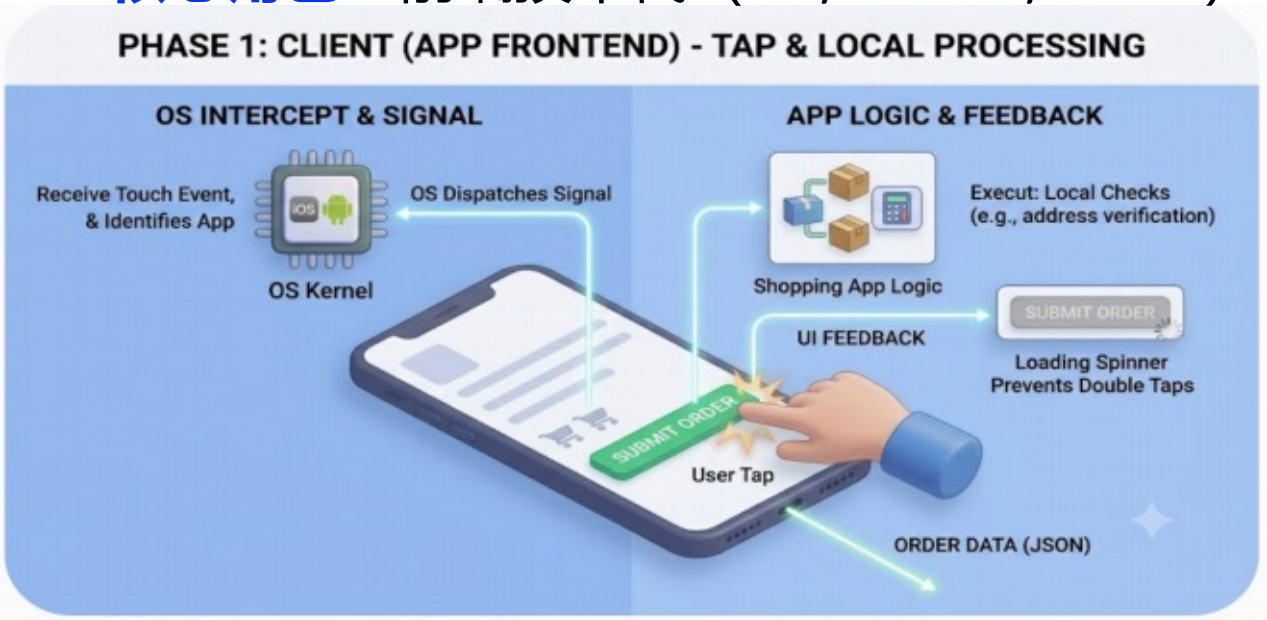
计算机系统理解：以客户端APP的一次请求为例



在一个网上购物App上点击“提交订单”按钮到显示订单处理结果的整个过程：
涉及软件客户端（App）、网络、软件服务端（后端服务）、数据库

第1阶段：软件客户端处理

核心角色：前端技术栈（iOS/Android/Flutter）



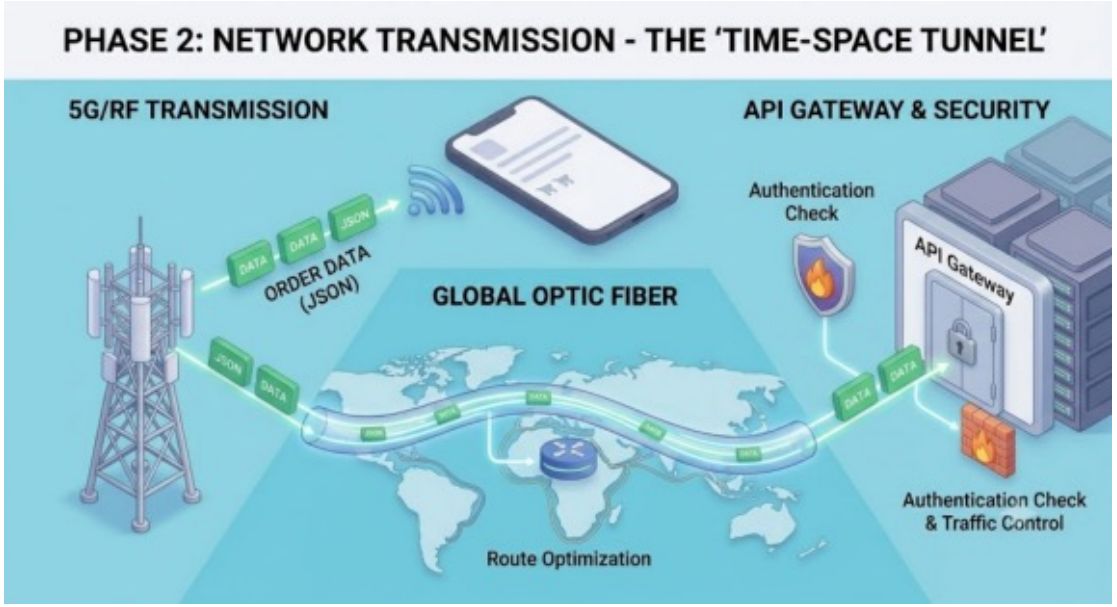
物理层到操作系统：屏幕捕捉到点击操作带来的电荷变化，手机操作系统（如Android）立刻翻译出点击坐标并转告购物App

App应用层（业务逻辑）： App前端代码开始运行，首先进行本地校验（如检查地址 是否提供、优惠券是否过期），确认无误后向服务器端发起请求并等待结果

视觉反馈： 为了不让用户感觉“死机”，App会在屏幕上显示转圈圈动画（Loading）

第2阶段：网络传输

核心角色：无线网络（5G/Wi-Fi）、HTTP协议、CDN/网关



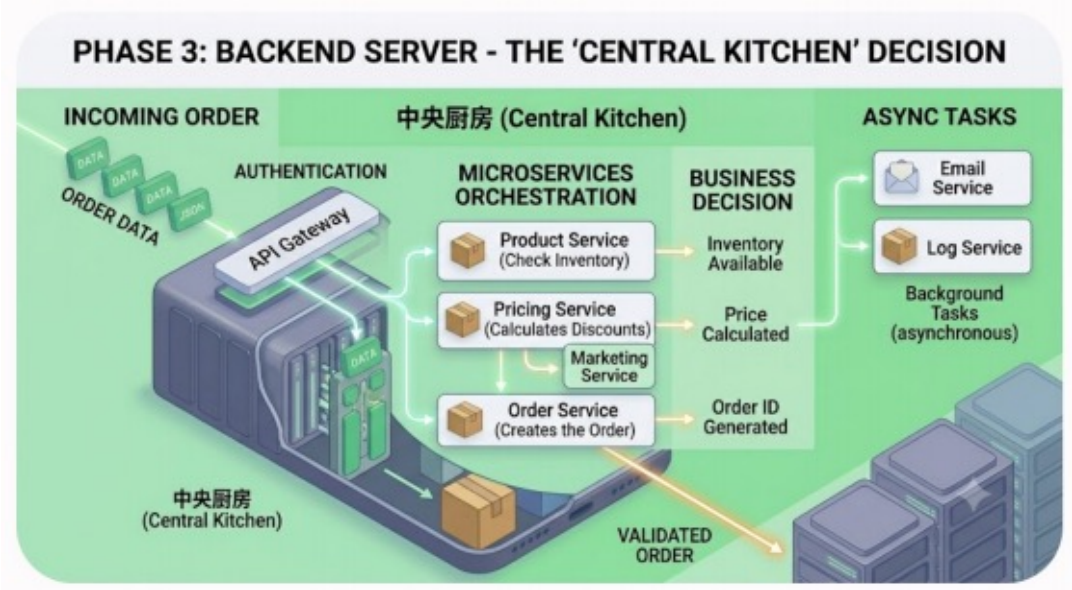
打包与发送：App把订单信息按照标准（HTTP/HTTPS协议）打包成一段格式化的文 本（如JSON），手机的无线芯片（5G或Wi-Fi）将这段文本发送到最近的通信基站

网络传输：数据进入运营商的骨干光纤网络，通过光缆传输，中途经过无数个路由器

服务端API网关接收：数据到达App服务端所在云服务器机房，首先由网关（Gateway）进行处理，像保安一样进行检查（是否合法用户请求）和流量控制（是否要等一等）

第3阶段：后端服务器处理

核心角色：后端技术栈（Java/Go/Python）、应用服务器



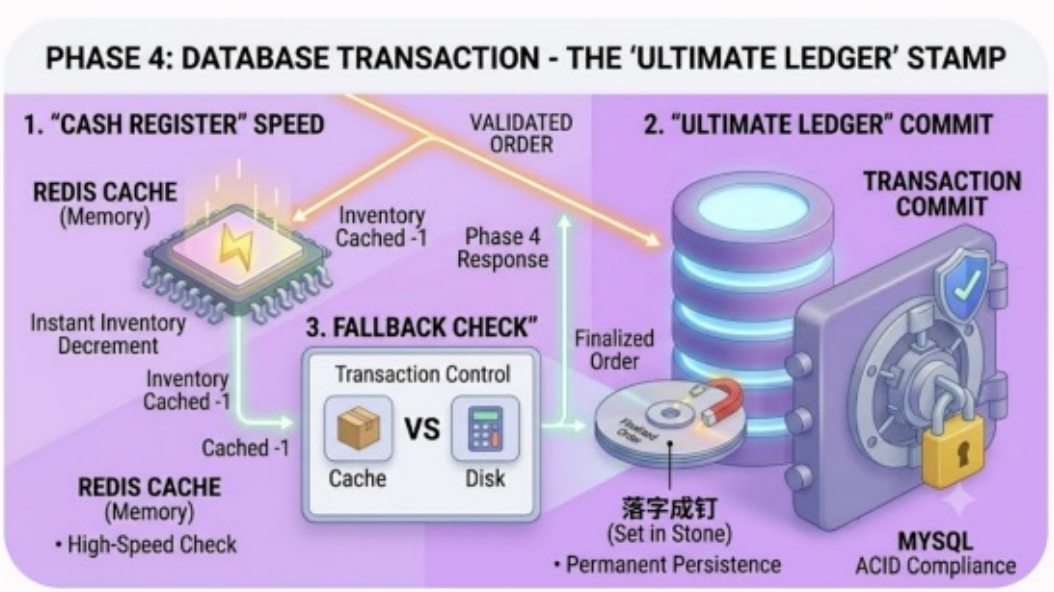
商品服务（对货）：服务器的大脑（CPU）开始运行Java或Go语言编写的代码，首先去核对：“仓库里真的还有这件衣服吗？”（检查库存）

营销服务（对账）：核对优惠券是否可用，满减是否正确，最终计算出实际支付金额

订单服务（创建）：确认无误后，生成一个唯一的“订单号”（如 20260611XXXX），并把订单状态标记为“待支付”

第4阶段：数据库持久化更新

核心角色：关系型数据库（MySQL）、缓存（Redis）

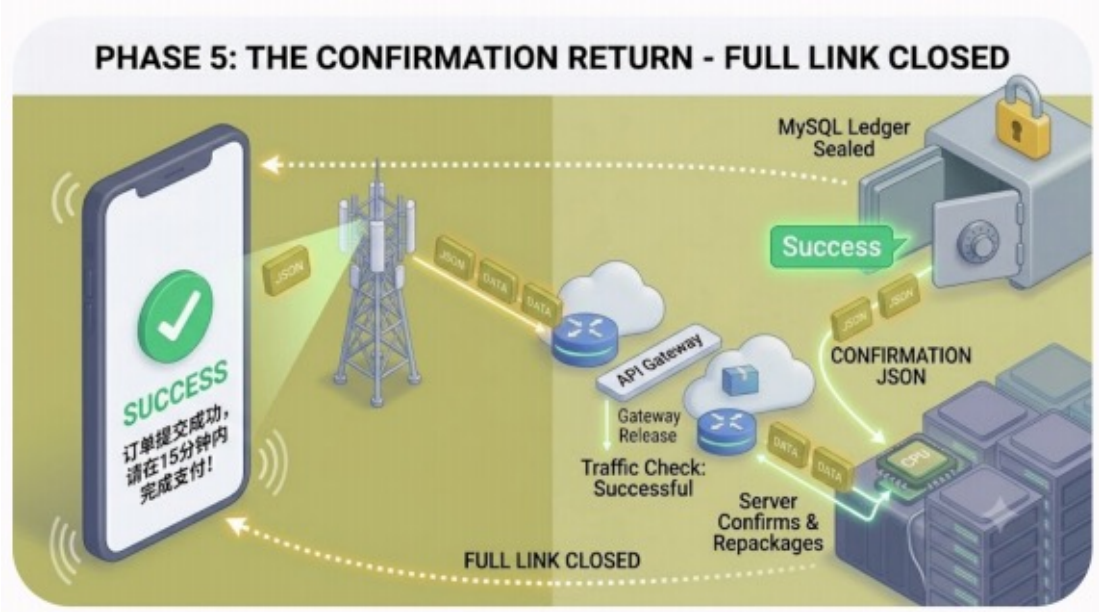


高并发的"小妙招"（缓存）：如果双十一期间10万人同时抢购，直接读写硬盘数据库会让系统瘫痪。服务器会先在内存数据库（如Redis）里把库存减1，速度极快。

严谨的"终极账本"（MySQL）：数据进入数据库并执行"事务（Transaction）"处理：要么扣减库存、保存订单两个动作同时成功；要么有一个失败，就全部退回原状（回滚）。绝对不能出现"库存扣了，订单没生成"的灾难。

第5阶段：响应信息原路返回

核心角色：全链路闭环



数据库处理完成：数据库通知后端服务器：“账本记好了！”

响应消息传回：后端服务器打包一个“成功”的JSON数据，通过网关和网络光纤原路发射回你的手机。手机的5G芯片收到信号，转交给购物App。

客户端App处理响应消息：App的动画停止转圈，屏幕上瞬间弹出一个漂亮的绿色对勾：“订单提交成功，请在15分钟内完成支付！”同时，手机轻轻一震。

软件设计思想

- 分解（Decomposition）与抽象（Abstraction）
- 软件体系结构（Software Architecture）
- 关注点分离（Separation of Concerns）
- 模块化（Modularity）
- 信息隐藏（Information Hiding）
- 重构（Refactoring）
- 复用（Reuse）

分解与抽象

- 分解和抽象是人类应对复杂性的两个基本手段，也是软件设计思维的重要基础
  - ✓ 分解：将软件不断地分解为更细粒度的代码单元，例如从组件、模块到 文件和类
  - ✓ 抽象：忽略无关细节，只保留与当前问题相关的关键信息，例如接口抽 象使我们可以针对接口进行编程而忽略内部数据结构、算法等实现细节
- 分解往往需要与抽象的思想相结合才能发挥应对复杂性的作用
- 数据抽象和过程抽象
  - ✓ 数据抽象：对于目标对象的数据化抽象描述
  - ✓ 过程抽象：对一系列过程性步骤和指令序列的整体抽象

软件抽象的选择

- 好的抽象应该屏蔽底层细节，突出事物的本质特性，同时符合人的思维方式，从而实现 降低复杂性的目标
- 由于针对抽象编程的实现方案不依赖于许多 无关细节，因此好的抽象还能极大提高程序 的可迁移性和可复用性
  - ✓ 例如，针对抽象的设备（例如抽象的打印机）编程的代码可以很容易地 与不同型号的设备（例如具体的打印机）一起工作，只要这些设备都能 实现同样的抽象设备接口（例如接受打印作业、返回打印机状态等）

抽象数据类型：更多的例子

List

Initialize list
Insert item in list
Remove item from list
Read next item from list

Stack

Initialize stack
Push item onto stack
Pop item from stack
Read top of stack

Light

Turn on
Turn off

Elevator

Move up one floor
Move down one floor
Move to specific floor
Report current floor
Return to home floor

模块化

- 模块化：将整个产品或系统分解为大小合适、相对独立的模块
  - ✓ 使得各个模块的生产制造可以相对独立地进行
  - ✓ 可以在不破坏整体结构的基础上实现模块替换和扩展
  - ✓ 适用于软件设计的各个层次，包括软件体系结构中的组件、组件内的类一级的详细设计
- 分解和抽象设计思想的集中体现
  - ✓ 模块结构给出的系统分解方案可以以分而治之的方式实现
  - ✓ 模块接口为外部调用和整体集成提供外部抽象
- 模块化设计强调模块独立性
  - ✓ 一般可以用内聚度和耦合度来衡量
  - ✓ 好的模块化设计应实现模块的高内聚和低耦合



信息隐藏

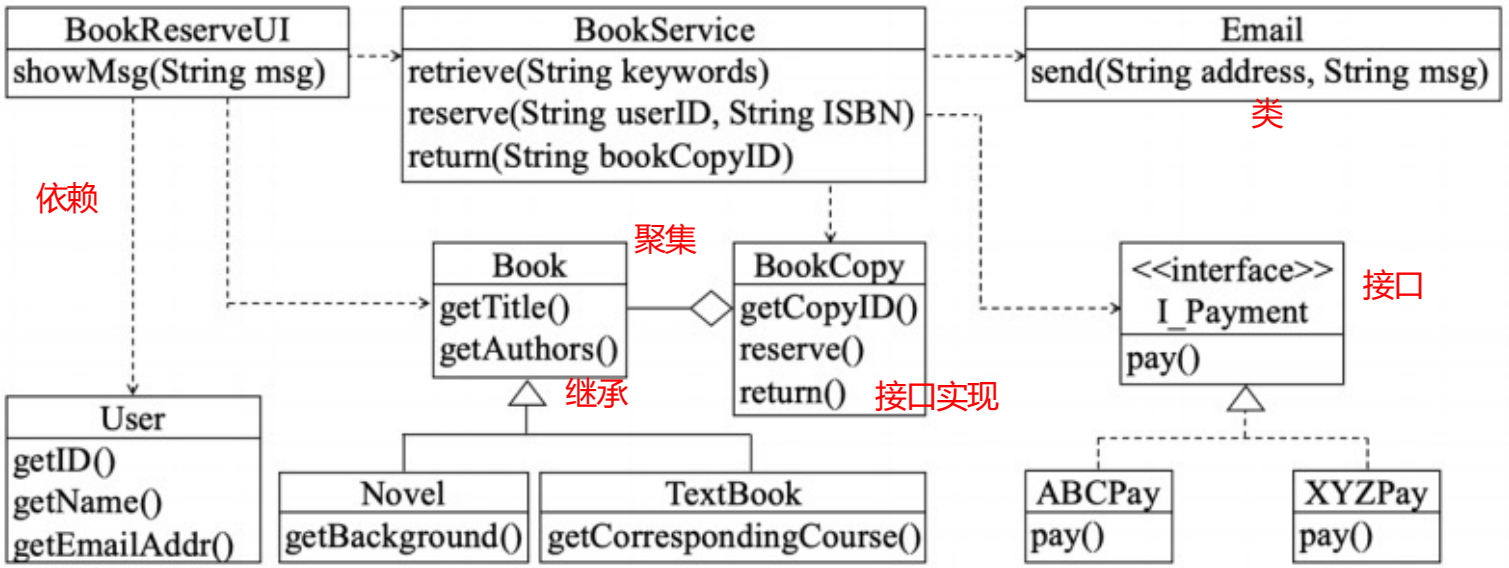
- 模块（例如组件或类）将实现细节隐藏在内部，仅通过受限的接口对外提供访问
- 如果将模块内部的实现细节都对外暴露，那么即使模块分解得当也会造成不必要的模块间耦合
- 面向对象软件设计与实现中的信息隐藏
  - ✓ 一个包（package）中作为“门面”（facade）让外部可见的一些类的访问修饰符可以设置为public，而其他对外隐藏的类可以设置为protected
  - ✓ 一个类（class）中作为对外接口一部分的属性和方法（一般建议属性不要直接对外开放）可以设置为public，而其他对外隐藏属性和方法可以设置为private

信息隐藏的好处

- 通过屏蔽实现细节以及暴露抽象接口的方式降低了其他模块开发者对于当前模块的认知复杂性
- 通过抽象降低了内部实现细节的变化对于其他模块的影响
- 通过受控接口提供访问，可以更好地实现对于内部数据和操作的保护

静态结构设计描述（UML类图）

校园一卡通系统中的图书馆管理子系统静态结构设计



类：表示为矩形组合，上下两部分分别是类名和方法（省去了属性、方法返回值和一部分方法）
接口：利用UML的构造型（stereotype）机制定义的一种特殊的“类”
关系：箭头表示类与类之间的各种静态结构关系

- ✓ 继承：类之间的继承表示一种泛化和特化关系（isA）
- ✓ 聚集：类之间的聚集关系表示一种整体和部分关系
- ✓ 依赖：类之间的依赖关系表示方法调用、属性访问等功能依赖

契约式设计（Design by Contract）

- 最早由Bertrand Meyer提出并在Eiffel语言中进行了实现
- 基本思想：建立不同类（或者模块）之间的契约关系，以一种可检查的方式明确定义接口的实现方和调用方各自应当承担的“权利”和“义务”
- 将接口的实现方和调用方之间的关系比喻成商业活动中供应方与需求方之间的关系
- 例如，在一个采购合同中
  - ✓ 供应方承诺交付的产品符合约定的质量要求同时按时交货，而其权利是 按照约定的时间和金额获得采购款
  - ✓ 需求方则承诺按照约定的时间和金额支付采购款，而其权利则是获得满 足所约定的质量要求的产品并且能按时收货

契约的类型

- 契约应当明确定义并且可以进行验证
- 常见的契约形式是对程序运行状态的断言，即关于输入参数、返回值、内部属性取值范围及其关系的布尔表达式
  - ✓ 前置条件（precondition）：调用一个方法之前应当满足的条件
  - ✓ 后置条件（postcondition）：一个方法执行完成后应当满 足的条件
  - ✓ 不变式（invariant）：一个软件元素（例如类或对象实例）在任何方法执行前后都应当一直满足的条件

堆栈类中的契约声明

```
public class IntStack{
  private int[] elements;
  private int topIndex;
  invariant
    topIndex_within_capacity: topIndex>=-1&&topIndex<elements.length;

  public IntStack(int capacity);
  require
    capacity_be_positive: capacity>0;
  ensure
    elements_length_equalTo_capacity: elements.length==capacity;

  public boolean isEmpty();
  ensure
    negative_topIndex_Indicate_Empty: Result==(topIndex<0);

  public int pop();
  require
    not_empty: isEmpty()==false;
  ensure
    return_top: Result==elements[old topIndex];
    topIndex_decrease: topIndex==old topIndex-1;
  …
}
```

不变式，堆栈对象任何方法执行前后都必须满足的

初始化方法的前置条件

初始化方法的后置条件

是否为空判断方法的后置条件

出栈方法的前置条件

出栈方法的后置条件

“编程”≠“软件开发”

经典教科书将软件定义为“程序+文档”， 但在**软件深入渗透人类社会经济生活以及社会物理空间方方面面**、 **“软件定义一切”的发展趋势**形成广泛共识的当下，我们有必要**重新审视“编程”与“软件开发”的区别**

为什么不一样

- 软件系统着眼于长期稳定运行并有效应对各种特殊情况
- 软件系统根植于复杂的社会物理环境之中
- 软件系统包含复杂而精巧的设计结构，各个部分密切配合
- 软件系统需要经历长期的演化和维护过程

为什么当下要强调这一点

- 数字化和信息化建设飞速发展，软件的规模和复杂性不断增长
- 软件定义一切的发展趋势更加显著，软件加速渗透到社会物理空间，同时系统的可靠性与安全性要求越来越高
- 随着大模型代码生成能力的不断增强，编程与软件开发之间的区分越来越明显，大模型擅长前者但从根本上缺少应对系统复杂性的手段

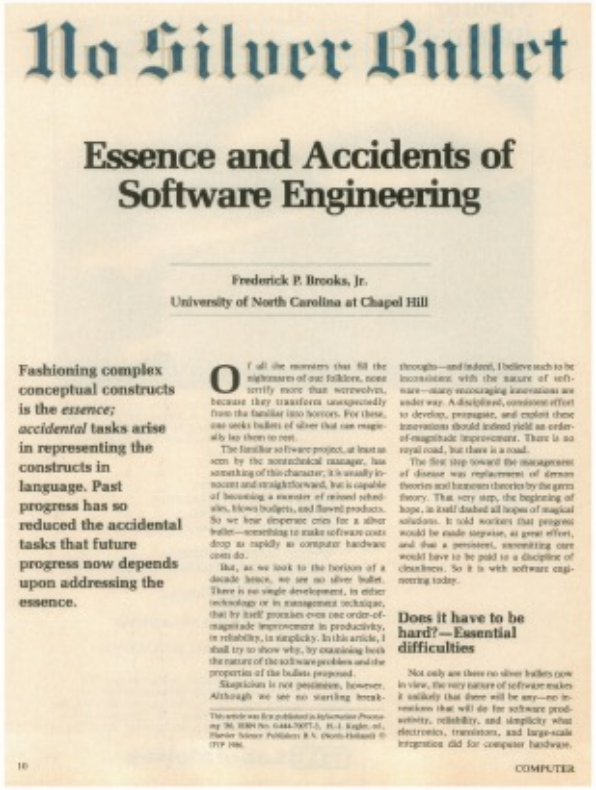
软件的三重角色和形态

形态	核心要求	持续演化要求	设计复杂度	主要挑战
解决方案	目标达成、成本、完成时间	无到低	无到低	意图理解、任务拆解、能力映射
工具应用	功能正确性、性能、易用性	低到中	低到中	需求分析、业务和数据设计、特性交互管理
复杂系统	功能正确性、性能、 可用性、安全性	中到高	中到高	系统理解、设计权衡决策、系统复杂性管理

为什么要在大模型和Agent等AI技术迅猛发展的当下强调这三种软件形态的区别？因为**AI技术对于这三种软件形态的实现能力存在着本质性的区别**

- **解决方案**：AI技术直接进行方案的规划、实现以及执行过程中的动态调整，甚至可以在运行时按需合成程序并执行，此时人工编程已经不再需要
- **工具应用**：AI技术基于用户需求描述生成较完善的软件Demo，或者在专家型开发者的加持下增量迭代地实现完整应用开发，此时人工编程已经成为辅助手段
- **复杂系统**：AI技术在缺少开发历史和设计知识的情况下，很难形成高层的系统理解并提供自动化开发支持，此时人类开发者的知识和经验仍然发挥着主导作用，而AI技术主要是提供一些辅助支持（如问题分析和修改决策建议以及局部的代码补全）

本质复杂性与偶然复杂性



No Silver Bullet

Essence and Accidents of Software Engineering

Frederick P. Brooks, Jr.
University of North Carolina at Chapel Hill

Fashioning complex conceptual constructs is the *essence*; *accidental* tasks arise in representing the constructs in language. Past progress has so reduced the accidental tasks that future progress now depends upon addressing the essence.

Of all the monsters that fill the nightmares of our folklore, none terrify more than werewolves, because they transform unexpectedly from the familiar into horrors. For these, one seeks bullets of silver that can magically lay them to rest.

The familiar software project, at least as seen by the nontechnical manager, has something of this character; it is usually innocent and straightforward, but is capable of becoming a monster of missed schedules, blown budgets, and flawed products. So we hear desperate cries for a silver bullet—something to make software costs drop as rapidly as computer hardware costs do.

But, as we look to the horizon of a decade hence, we see no silver bullet. There is no single development, in either technology or in management technique, that by itself promises even one order-of-magnitude improvement in productivity, in reliability, in simplicity. In this article, I shall try to show why, by examining both the nature of the software problem and the properties of the bullets proposed.

Skepticism is not pessimism, however. Although we see no startling breakthroughs—and indeed, I believe such to be inconsistent with the nature of software—many encouraging innovations are under way. A disciplined, consistent effort to develop, propagate, and exploit these innovations should indeed yield an order-of-magnitude improvement. There is no royal road, but there is a road.

The first step toward the management of disease was replacement of demon theories and humours theories by the germ theory. That very step, the beginning of hope, in itself dashed all hopes of magical solutions. It told workers that progress would be made stepwise, as germ theory succeeded theory, and that the sources of the progress would be many.

Does it have to be hard?—Essential difficulties

Not only are there no silver bullets now in view, the very nature of software makes it unlikely that there will be any—no inventions that will do for software productivity, reliability, and simplicity what electronics, transistors, and large-scale integration did for computer hardware.

This article was first published in Information Processing 86, ISBN No. 0444-70077-3, H.-J. Kugler, ed., Elsevier Science Publishers B.V. (North-Holland) © IFIP 1986.

Frederick P. Brooks. No Silver Bullet: Essence and Accidents of Software Engineering. Computer, Volume: 20, Issue: 4, April 1987.

本质复杂性

- 来源于问题域本身的、固有的、不可简化的复杂性，是解决问题所必须面对的、内在的困难
- 例如，一个软件所需要实现的业务逻辑以及所处的外部环境的复杂性都属于本质复杂性

偶然复杂性

- 并非问题本身所必需，而是在解决过程中，由于所选择的工具、方法、技术、环境或架构缺陷等带来的、附加的复杂性
- 例如，一个软件在设计和实现时所采取的具体策略，以及由于理解和经验不足导致的模块化设计不足、代码逻辑混乱等问题

本质和偶然复杂性增强的原因

本质复杂性：现实世界和社会物理空间加速向软件和数字世界映射

- **“软件定义一切”趋势下软件应用领域扩展加深**：软件应用领域持续扩展和加深，规 模不断扩大，各个行业领域的复杂问题正越来越多地通过软件来表达和实现
- **“人机物融合万物互联”趋势下的软件系统互联与融合**：软件连接的设备和人的范围 越来越大，同时信息系统正在加速融合，系统之系统正变得无处不在
- **软件系统中的社会性与价值观投射不断加强**：随着软件对于社会经济生活方方面面的 加速渗透，人类社会中的各种复杂问题以及人类价值观也会不断向软件系统中投射

偶然复杂性：问题空间与解空间的巨大鸿沟以及人类的思维局限性

- **软件开发中的阻抗失配与不完美抽象**：关于软件的高层思考以一种不协调的方式杂糅到代码中，不完美的抽象体系导致高层思想在底层实现中产生意想之外的碰撞冲突
- **开发人员的思维局限性与有限责任心**：开发人员经常是在有限理解的情况下做出设计和实现决策，同时并不总是能够保证以最强的责任心进行问题的推敲和方案的规划
- **复杂系统的涌现效应**：软件系统的组成成分及相互影响非常复杂，由此导致很多细微的相互作用以一种意想不到的方式传播和积聚，最终产生某种较大的潜在影响

生成式软件开发

End

11. 从大众开发到专业开发